



CY Tech — Préparation ING1
Informatique II — Semestre 2 — 2025/2026

Projet **CY-biblioTECH**

Rapport de projet

Membres du groupe

Zineb Glizi · Myriam Ziani · Yasser Torkhani

Encadrants : Eva Ansermin — Romuald Grignon
eva.ansermin@cyu.fr — romuald.grignon@cyu.fr

1. Introduction

Le projet CY-biblioTECH est un programme développé en langage C dans le cadre du cours d'Informatique II de première année à CY Tech. L'objectif principal est de concevoir et d'implémenter un système complet de gestion d'une bibliothèque universitaire, entièrement piloté en ligne de commande.

Le programme permet de gérer deux types d'utilisateurs — les étudiants et les professeurs — avec des droits différents en matière d'emprunts. Il gère un catalogue de livres, les emprunts et les retours, tout en appliquant les règles propres à la bibliothèque. L'ensemble des données est persisté dans des fichiers texte lisibles, rechargés automatiquement à chaque lancement.

Ce projet nous a permis de mettre en pratique plusieurs notions fondamentales du cours : les structures en C, la gestion des fichiers, les chaînes de caractères, la modularité du code, et la gestion du temps avec les timestamps Unix.

2. Organisation de l'équipe

Le groupe est composé de trois membres : Zineb Glizi, Myriam Ziani et Yasser Torkhani. Dès le démarrage du projet, nous avons choisi de nous répartir les tâches de manière progressive, en construisant le programme étape par étape. Chaque partie du code a été développée par un membre du groupe, puis partagée et expliquée aux autres afin que tout le monde comprenne l'ensemble du projet.

Cette approche nous a permis de travailler efficacement en équipe : chaque membre savait ce que les autres avaient fait, pouvait relire le code, proposer des améliorations et continuer à construire dessus. Nous avons également effectué des recherches individuelles sur les points techniques que nous ne maîtrisions pas encore (notamment la gestion du temps avec `time.h`, le tri de tableaux de structures, et la lecture de fichiers avec commentaires).

3. Étapes de développement

Étape 1 — Référence de départ (Zineb)

Avant de commencer le projet à proprement parler, Zineb a repris le code rédigé lors du CC1 du deuxième semestre, dont le sujet portait également sur la gestion d'une bibliothèque. Ce travail préalable a servi de base de référence : les structures `Date` et `Livre`, les fonctions de comparaison de dates et d'ajout de jours, ainsi que les fonctions de sauvegarde et de chargement par fichier y étaient déjà esquissées. Cela a permis d'avoir un point de départ solide avant d'aborder les fonctionnalités spécifiques au projet.

Étape 2 — Chargement des livres (Zineb)

Zineb a ensuite travaillé sur le module de gestion des livres, en écrivant les fonctions `charger_livres()` et `sauver_livres()` dans le fichier `livre.c`.

Le format choisi pour `books.txt` est un format texte lisible, avec 4 lignes par livre : l'identifiant, le titre, l'auteur et la catégorie. Pour rendre le fichier plus clair et maintenable, les lignes vides et les lignes commençant par `#` (commentaires) sont ignorées à la lecture. Une fonction dédiée `lire_ligne_utile()` a été développée dans `main.c` pour centraliser cette logique de filtrage.

Le tableau global livres[] et son compteur nb_livres sont déclarés dans livre.c et exposés aux autres modules via extern dans livre.h. Cette organisation permet à emprunt.c d'accéder aux livres sans dupliquer les données.

Étape 3 — Connexion des utilisateurs (Myriam)

Myriam a pris en charge le module utilisateur, en commençant par la fonctionnalité de connexion. La structure Utilisateur contient un login, un mot de passe et un rôle (caractère 'E' pour étudiant ou 'P' pour professeur). Les utilisateurs sont stockés dans users.txt selon le même format que les livres : 3 lignes par utilisateur, commentaires ignorés.

La fonction connexion() demande le login et le mot de passe, parcourt le tableau users[] pour vérifier la correspondance, et retourne l'indice de l'utilisateur trouvé (ou -1 en cas d'échec). Ce retour d'indice est utilisé dans main.c pour afficher le menu personnalisé selon le rôle.

Étape 4 — Inscription (Zineb)

Zineb a complété le module utilisateur en ajoutant la fonctionnalité d'inscription. La fonction inscription() vérifie d'abord que le login saisi n'est pas déjà pris (via trouver_user()), puis demande le mot de passe et le rôle. Elle convertit automatiquement le rôle en majuscule pour être robuste aux saisies comme 'e' ou 'p', et rejette tout rôle invalide. Le compte est ensuite ajouté au tableau en mémoire et sauvegardé immédiatement dans users.txt.

Étape 5 — Emprunter et rendre (Myriam)

Myriam a développé les fonctions principales du module d'emprunt dans emprunt.c. Un emprunt est représenté par la structure Emprunt, qui contient l'identifiant du livre, le login de l'emprunteur, et l'heure d'emprunt sous forme de timestamp Unix (valeur entière retournée par time(NULL), représentant le nombre de secondes écoulées depuis le 1er janvier 1970).

La fonction emprunter() commence par vérifier les règles avant d'afficher quoi que ce soit : l'utilisateur a-t-il des livres en retard ? A-t-il atteint sa limite ? Le tableau global est-il plein ? Si tout est correct, elle propose les options de tri et d'affichage, puis enregistre l'emprunt.

La fonction rendre() demande l'identifiant du livre à rendre, vérifie que l'utilisateur l'a bien emprunté, puis supprime l'emprunt du tableau en décalant les éléments suivants d'une case vers la gauche — une technique classique de suppression dans un tableau non trié.

Étape 6 — Gestion des deadlines (Yasser)

Yasser a ajouté la gestion des échéances de rendu. La durée d'emprunt est définie par des constantes dans emprunt.h : 120 secondes pour les étudiants (DUREE_ETU) et 180 secondes pour les professeurs (DUREE_PROF). La fonction duree_pour() retourne la durée adaptée selon le rôle de l'utilisateur.

La fonction afficher_mes_emprunts() calcule pour chaque livre emprunté le temps restant avant l'échéance (deadline - maintenant). Si cette valeur est négative, le livre est affiché comme étant en retard avec le nombre de secondes de dépassement. Cette information est systématiquement affichée à l'utilisateur dès qu'il se connecte.

Étape 7 — Application des règles (Myriam)

Myriam a ensuite implémenté la vérification des règles de la bibliothèque dans la fonction emprunter(), en s'appuyant sur les fonctions a_des_retards() et compter_emprunts_user() développées au module précédent.

- Un étudiant peut emprunter au maximum 3 livres simultanément (MAX_ETUDIANT = 3).
- Un professeur peut en emprunter jusqu'à 5 (MAX_PROF = 5).

- Tout utilisateur ayant au moins un livre en retard ne peut pas effectuer de nouvel emprunt, quel que soit son rôle.
- Il est impossible d'emprunter un livre déjà pris par quelqu'un d'autre, vérifié par la fonction `livre_emprunte()`.

Ces vérifications sont effectuées en tout début de la fonction `emprunter()`, avant même d'afficher le menu de recherche, afin d'éviter toute interaction inutile avec un utilisateur qui ne peut pas emprunter.

Étape 8 — Tri et filtre par catégorie (Yasser)

Yasser a implémenté la fonction `trier_livres()` dans `livre.c`, qui trie le tableau global `livres[]` selon trois modes différents : par titre ('t'), par auteur ('a') ou par catégorie ('c'). Le tri utilisé est un tri à bulles, algorithme simple et suffisant pour les volumes de données attendus dans ce contexte.

Le principe du tri à bulles consiste à parcourir le tableau en comparant deux éléments adjacents et à les échanger s'ils sont dans le mauvais ordre, en répétant l'opération jusqu'à ce que le tableau soit entièrement trié. Pour comparer les chaînes de caractères, on utilise `strcmp()` de `string.h`, qui retourne une valeur positive si la première chaîne est alphabétiquement après la seconde.

Dans la fonction `emprunter()`, lorsque l'utilisateur choisit de filtrer par catégorie, il saisit le nom de la catégorie souhaitée. Seuls les livres disponibles correspondant exactement à cette catégorie sont alors affichés. Le tri est appliqué sur le tableau en mémoire, puis immédiatement sauvegardé dans `books.txt` via `sauver_livres()`, de sorte que l'ordre soit conservé entre les sessions.

Étape 9 — Découpage en modules (Zineb) et Makefile (Myriam)

Une fois le programme fonctionnel dans un fichier unique, Zineb a procédé au découpage du code en plusieurs fichiers distincts, conformément à l'exigence de modularité du cahier des charges.

Le projet final est organisé en trois modules indépendants :

- `livre.h / livre.c` — gestion du catalogue de livres
- `utilisateur.h / utilisateur.c` — gestion des comptes et de l'authentification
- `emprunt.h / emprunt.c` — gestion des emprunts et application des règles

Chaque fichier `.h` expose uniquement les types et prototypes nécessaires aux autres modules (via `extern` pour les variables globales), ce qui évite les inclusions circulaires et garantit une compilation propre.

Les fonctions utilitaires partagées par tous les modules (`enlever_retour_ligne`, `vider_buffer`, `lire_ligne_utile`) ont été définies dans `main.c` et déclarées en haut de chaque fichier `.c` qui les utilise, sans passer par un header dédié.

Myriam a ensuite rédigé le script `test.sh` qui compile l'ensemble des fichiers avec `gcc` et lance directement le programme, simplifiant ainsi la procédure de test pour toute l'équipe.

4. Difficultés rencontrées et solutions

Gestion du buffer d'entrée

L'un des problèmes les plus fréquents lors du développement a été la gestion du tampon d'entrée standard après un `scanf()`. Lorsqu'on lit un entier avec `scanf("%d", ...)`, le caractère de retour à la ligne reste dans le buffer et est immédiatement consommé par le `fgets()` suivant, ce qui provoque des saisies vides. Nous avons résolu ce problème en appelant systématiquement `vider_buffer()` après chaque `scanf()`.

Lecture des fichiers avec commentaires

Pour rendre les fichiers books.txt, users.txt et loans.txt lisibles et annotables, nous avons choisi un format avec lignes de commentaires (préfixées par #) et lignes vides. La difficulté était de faire en sorte que toutes les fonctions de chargement sautent ces lignes correctement. Nous avons centralisé cette logique dans une seule fonction lire_ligne_utile() pour éviter de dupliquer le code dans chaque module.

Timestamps Unix

La gestion des échéances d'emprunt reposant sur des timestamps Unix (time_t) n'était pas un concept familier. Après quelques recherches sur la documentation de time.h, nous avons compris que time(NULL) retourne le nombre de secondes depuis le 1er janvier 1970, et qu'il suffit d'y ajouter la durée en secondes pour obtenir l'échéance. La comparaison avec l'heure actuelle devient alors une simple soustraction.

Suppression dans un tableau

La suppression d'un emprunt dans le tableau emprunts[] sans utiliser de liste chaînée nécessitait de décaler tous les éléments suivants d'une position vers la gauche. Cette technique, bien que simple, doit être appliquée soigneusement pour ne pas décaler un élément de trop ou laisser des doublons.

5. Conclusion

Le projet CY-biblioTECH nous a permis de développer une application complète en langage C, depuis la conception des structures de données jusqu'à la persistance sur fichier, en passant par la gestion des interactions utilisateur et l'application de règles métier.

L'organisation en équipe avec une répartition claire des tâches a été efficace : chaque étape s'est construite sur la précédente, et le fait d'expliquer son code aux autres membres a renforcé notre compréhension de l'ensemble du projet. Nous avons également appris à lire de la documentation (time.h, string.h), à déboguer des problèmes liés au buffer d'entrée, et à structurer un projet C en plusieurs fichiers.

Le programme final répond à l'ensemble des exigences du cahier des charges : gestion de deux types d'utilisateurs, catalogue de livres avec tri et filtre, emprunts avec vérification des règles, deadlines, et sauvegarde persistante dans des fichiers texte commentés et lisibles.